

07 — Funktionen

Eigene Werkzeuge bauen mit def, Parametern und return

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Funktionen? – das DRY-Prinzip
2. def – eigene Funktionen definieren
3. Unter der Haube: Definition $\neq$ Ausführung
4. Parameter und Argumente
5. Unter der Haube: was beim Aufruf passiert
6. return – Werte zurückgeben (vs. print)
7. Scope: lokal vs. global
8. **Klassiker: Umrechner-Bibliothek** – live programmiert

Lernziel: Eigene Funktionen schreiben – und **verstehen**, was Python bei def und bei jedem Aufruf im Speicher tut.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 07).

Wozu Funktionen?

Ohne Funktion

```
print(f"Eco-Sneaker kostet 89.95 EUR")  
print(f"Hemp-High kostet 109.00 EUR")  
print(f"Bambus-Boot kostet 135.50 EUR")
```

Mit Funktion

```
def zeige_produkt(name, preis):  
    print(f"{name} kostet {preis} EUR")
```

```
zeige_produkt("Eco-Sneaker", 89.95)
```

DRY: *Don't Repeat Yourself* – die Logik steht **einmal**. Ändert sich das Format, ändern wir **eine** Zeile. Neu ist heute: wir bauen eigene **Werkzeuge** mit eigenem Namen.

Eingebaute Funktionen kennen Sie schon

```
len(["Eco-Sneaker", "Hemp-High"])  # ergibt 2
type(89.95)                        # ergibt <class 'float'>
input("Name: ")                    # liest Eingabe, gibt sie her
```

- Aufruf = **Name + runde Klammern**, darin die **Argumente**
- Manche liefern einen **Wert** zurück (len, type, input), andere zeigen nur an (print)
- Heute lernen wir, solche Bausteine **selbst zu bauen**

 **Sofort ausprobieren** – Notebook 07, Kap. 1: drei Aufgaben mit len, type, input

def - Anatomie einer Funktion

```
def zeige_shop_info():  
    print("Willkommen bei Veggie Soles!")  
    print("Vegane Sneaker, fair produziert.")
```

`zeige_shop_info()` *# Aufruf -- beliebig oft*

- `def` + **Name** (Snake-Case) + `()` + `:`
- **Einrückung** (4 Leerzeichen) markiert den Rumpf
- Definition = Rezept aufschreiben, Aufruf = kochen

► **Live-Demo** - 01_einfache_funktion.py

📝 **Sofort ausprobieren** - Notebook 07, Kap. 2: `zeige_info`, `trennlinie`, `zeige_liste`

Unter der Haube: def erzeugt ein Objekt

```
def netto_zu_brutto(netto):  
    return netto * 1.19      # laeuft JETZT nicht!  
  
print(type(netto_zu_brutto)) # <class 'function'>
```

Zeitpunkt	Speicher (Name → Objekt)
nach def	netto_zu_brutto → Funktionsobjekt
Rumpf schon gelaufen?	nein – er ist nur der Bauplan
erst netto_zu_brutto(100)	führt den Rumpf aus

Wie in Foliensatz 04: ein Name zeigt auf ein Objekt. Neu: dieses Objekt ist **ausführbarer Code** – die Klammern () starten ihn.

Parameter - die Stellschrauben

```
def zeige_produkt(name, preis):  
    print(f"{name}: {preis} EUR")
```

```
zeige_produkt("Eco-Sneaker", 89.95)
```

```
zeige_produkt("Hemp-High", 109.00)
```

- **Parameter** = Platzhalter in der Definition (name, preis)
- **Argumente** = konkrete Werte beim Aufruf
- Gleiche Funktion, unterschiedliche Eingaben – das macht sie wiederverwendbar

Positional vs. benannt

Positional - Reihenfolge zählt

```
zeige_produkt("Eco-Sneaker", 89.95)      # ok  
zeige_produkt(89.95, "Eco-Sneaker")     # Unfug!
```

Benannt - Reihenfolge egal

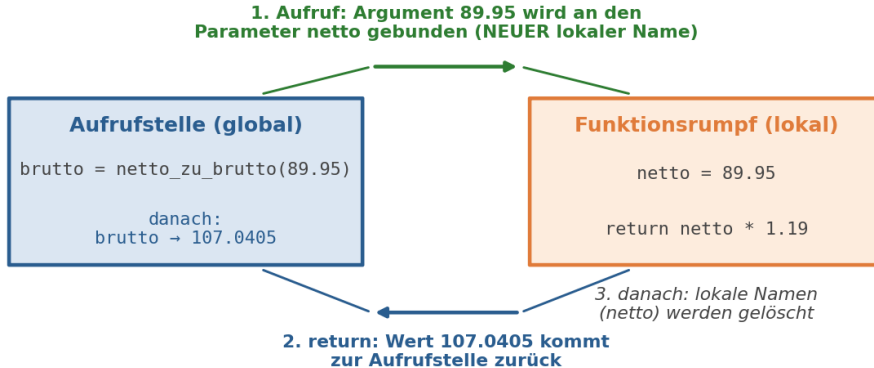
```
zeige_produkt(name="Eco-Sneaker", preis=89.95)  
zeige_produkt(preis=89.95, name="Eco-Sneaker")
```

Mischung erlaubt, **aber**: erst positional, dann benannt –
`zeige_produkt("Eco-Sneaker", preis=89.95).`

► **Live-Demo** - 02_funktion_mit_parametern.py

✎ **Sofort ausprobieren** - Notebook 07, Kap. 3: rechne_rechteck, zeige_produkt (positional + benannt)

Unter der Haube: was ein Aufruf wirklich tut



Der Rumpf ist eine **eigene kleine Welt**: Argumente gehen als **neue lokale Namen** hinein, return bringt genau **einen Wert** heraus – danach wird die Welt abgeräumt.

Unter der Haube: der Aufruf als Trace

```
def netto_zu_brutto(netto):  
    return netto * 1.19
```

```
brutto = netto_zu_brutto(89.95)
```

Schritt	Was Python tut	Speicher danach
1	def: Bauplan anlegen	netto_zu_brutto → Funktion
2	Aufruf: netto = 89.95 (neu, lokal)	lokal: netto
3	Rumpf: 89.95 * 1.19	Wert 107.0405
4	return: Wert zurück, lokal weg	brutto → 107.0405

Trace-Tabellen wie diese sind Ihr Werkzeug, um **jeden** Aufruf nachzuvollziehen – Schritt für Schritt auf Papier.

return - Werte zurückgeben

```
def netto_zu_brutto(netto):  
    return netto * 1.19
```

```
brutto = netto_zu_brutto(89.95)  
print(f"Brutto: {brutto:.2f} EUR")    # 107.04
```

- return beendet den Aufruf **sofort** und liefert den Wert an die Aufrufstelle
- Der Wert ist **weiterverwendbar**: speichern, weiterrechnen, formatieren
- Ohne return liefert eine Funktion keinen Wert

► **Live-Demo** - 03_funktion_mit_return.py

Unter der Haube: return vs. print

```
def mit_print(netto):  
    print(netto * 1.19)    # zeigt nur an
```

```
def mit_return(netto):  
    return netto * 1.19    # gibt Wert zurueck
```

	x = mit_print(100)	x = mit_return(100)
Konsole zeigt	119.0	nichts
x ist danach	None (kein Wert)	119.0
weiterrechnen	unmöglich	x * 2 usw.

print ist ein **Seiteneffekt** Richtung Bildschirm. return ist das **Ergebnis** – nur damit kann Ihr Programm weiterarbeiten.

Unter der Haube: eine Funktion, mathematisch

$$f : \text{Eingaben} \rightarrow \text{Ausgabe}, \quad f(\text{netto}) = 1.19 \cdot \text{netto}$$

- netto_zu_brutto ist genau dieses f : jeder Eingabe wird **ein** Ergebnis zugeordnet
- **Determinismus** (Foliensatz 01): gleiche Eingabe $\rightarrow$ garantiert gleiche Ausgabe
- Solche rechnenden Funktionen sind leicht zu prüfen: Wertetabelle genügt – mehr dazu bei den Algorithmen (NB 13a)

 **Sofort ausprobieren** – Notebook 07, Kap. 4: berechne_kreis, erstelle_email

Mehrere Rückgabewerte

```
def warenkorb_info(preise):  
    return len(preise), sum(preise)
```

```
korb = [89.95, 109.00, 135.50]  
n, summe = warenkorb_info(korb)  
print(f"{n} Artikel, {summe:.2f} EUR")
```

- Python packt die Werte in ein **Tupel** (Foliensatz 06)
- Beim Aufruf in mehrere Variablen **entpacken**

► **Live-Demo** - 04_mehrere_rueckgaben.py

📝 **Sofort ausprobieren** - Notebook 07, Kap. 4: analysiere_wort

Scope: wo gilt ein Name?

```
def netto_zu_brutto(netto):  
    rate = 0.19                      # lokal  
    return netto * (1 + rate)  
  
brutto = netto_zu_brutto(100)  
print(rate)                          # NameError!
```

- rate und netto sind **lokal**: sie entstehen beim Aufruf und werden nach return **entsorgt** (Folie "Aufruf-Ablauf")
- Deshalb stören sich Funktionen nie gegenseitig – jeder Aufruf hat seine eigene kleine Welt

Unter der Haube: Namensauflösung

```
MWST = 0.19          # global (Konvention: GROSS)
```

```
def netto_zu_brutto(netto):  
    return netto * (1 + MWST)  # woher kommt MWST?
```

Python sucht MWST ...	Treffer?
1. lokal (im laufenden Aufruf)	nein
2. global (oben im Programm)	ja: 0.19
3. eingebaut (print, len, ...)	-

Regel: **lokal** → **global** → **eingebaut**, der erste Treffer gewinnt. Nirgends gefunden? **NameError**.

► **Live-Demo** - 05_scope_demo.py

📎 **Sofort ausprobieren** - Notebook 07, Trainingsmaterial Aufg. 9 ("Scope verstehen")

Klassiker: Umrechner-Bibliothek - die Aufgabe

In Foliensatz 05 haben wir **eine** Temperatur umgerechnet - jetzt bauen wir das **Werkzeug** dazu:

- `celsius_zu_fahrenheit(c)` - Formel $F = C \cdot 9/5 + 32$
- `fahrenheit_zu_celsius(f)` - die Umkehrung
- `netto_zu_brutto(netto)` - Bruttopreis bei 19 % MwSt
- Alle drei geben Werte **zurück** (return, kein print im Rumpf) - Aufrufe mit Veggie-Soles-Werten

So soll es laufen

21.5 Grad C sind 70.70 Grad F

89.95 EUR netto sind 107.04 EUR brutto

► **Live-Coding ab leerem Editor** - Referenz: 90_klassiker_umrechner.py
Teilziele:

1. `def celsius_zu_fahrenheit(celsius)` - `def` + Parameter
2. `return` statt `print` im Rumpf
3. Aufruf, Ergebnis in Variable, formatiert ausgeben
4. zweite Funktion nach gleichem Muster
5. Funktionen kombinieren: Rückprobe $f(g(x)) \approx x$

✎ **Zum Selberlösen** - Handout "*Klassiker 07: Umrechner-Bibliothek*" (Klassiker/07_Klassiker_Umrechner.pdf). Vertiefung: Notebook 07, Trainingsmaterial.

Cheat Card

Aufgabe	Syntax
Funktion definieren	<code>def name():</code>
mit Parametern	<code>def name(p1, p2):</code>
Wert zurückgeben	<code>return wert</code>
mehrere Werte	<code>return a, b</code>
Ergebnis speichern	<code>x = name(arg)</code>
benannt aufrufen	<code>name(p2=w2, p1=w1)</code>
Klassiker heute	<code>3 × def + return + Aufrufe</code>

Faustregel: eine Funktion = **eine** Aufgabe. Im Rumpf wird **gerechnet** (return) – das print gehört an die Aufrufstelle.

Ausblick: Notebook 08 - Operatoren

Unsere Umrechner-Bibliothek rechnet **alles** klaglos um:

```
celsius_zu_fahrenheit(-500)    # unter dem absoluten  
                                # Nullpunkt!  
netto_zu_brutto(-89.95)        # negativer Preis?
```

Um Eingaben zu **prüfen**, fehlen uns Werkzeuge für Bedingungen:

- Vergleiche: == != < > <= >=
- Logik: and or not – als **Boolesche Algebra** mit Rechenregeln
- Zugehörigkeit: in / not in auf Listen und Dicts

Danach (NB 09) verzweigen wir damit: *wenn ungültig, dann Fehlermeldung.*

Heute geübt

- ✓ DRY-Prinzip: Logik einmal schreiben, oft aufrufen
- ✓ `def` erzeugt ein Funktionsobjekt – Definition $\neq$ Ausführung
- ✓ Aufruf-Ablauf: Argument $\rightarrow$ lokaler Parameter $\rightarrow$ Rumpf $\rightarrow$ return
- ✓ `return` vs. `print`: Ergebnis vs. Seiteneffekt
- ✓ Funktion als mathematisches $f(x)$: gleiche Eingabe, gleiche Ausgabe
- ✓ Scope: lokal $\rightarrow$ global $\rightarrow$ eingebaut
- ✓ **Klassiker Umrechner-Bibliothek** live gesehen

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Umrechner-Bibliothek** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 07
- Trainingsmaterial: 9 Aufgaben in drei Schwierigkeitsstufen